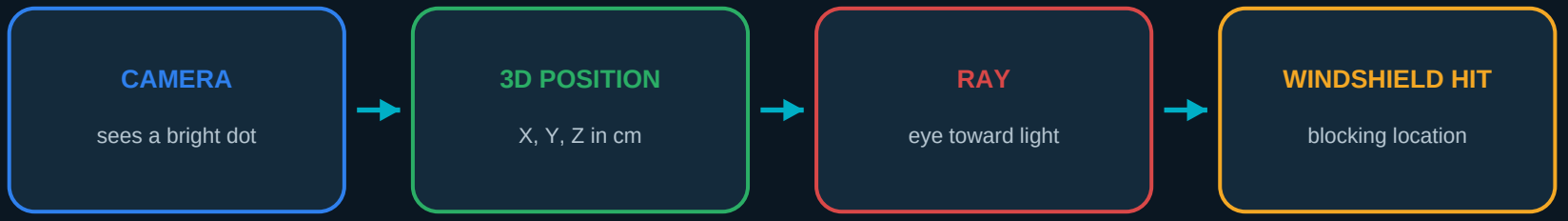
VISUAL EXPLAINER

How GlareTracker Calculates Headlight Glare

A picture-first guide to finding a headlight, estimating where it is, and calculating the exact place its light ray crosses the windshield.



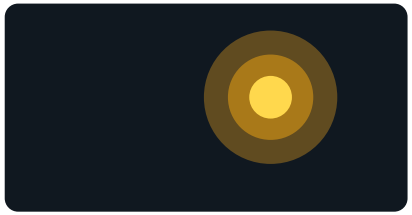
Audience: age 15+ | Based directly on the current GlareTracker source code

STEP 1

Turn a camera image into one reliable bright point

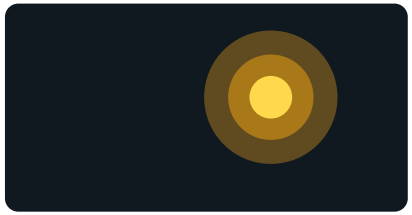
The computer does not understand 'headlight' first. It starts by finding a very bright group of pixels.

1. Camera frame



A dark road image contains one intense light.

2. Clean image



Convert to gray and blur 5 x 5 to reduce tiny noise.

3. Keep bright pixels

Threshold 240/255, then remove small specks.

4. Find center

Choose the largest valid blob and calculate its center.

WHY A WEIGHTED CENTER?

A headlight is not a perfect single pixel. Brighter pixels receive more 'votes', so the calculated center lands closer to the true light core. The code accepts a blob only when its area is between 5 and 10,000 pixels.

IDEA

center = sum(pixel x brightness) / sum(brightness)

The same idea is used for horizontal and vertical position.

Source: glare_tracker/headlight_detector.py - detect_bright_spot()

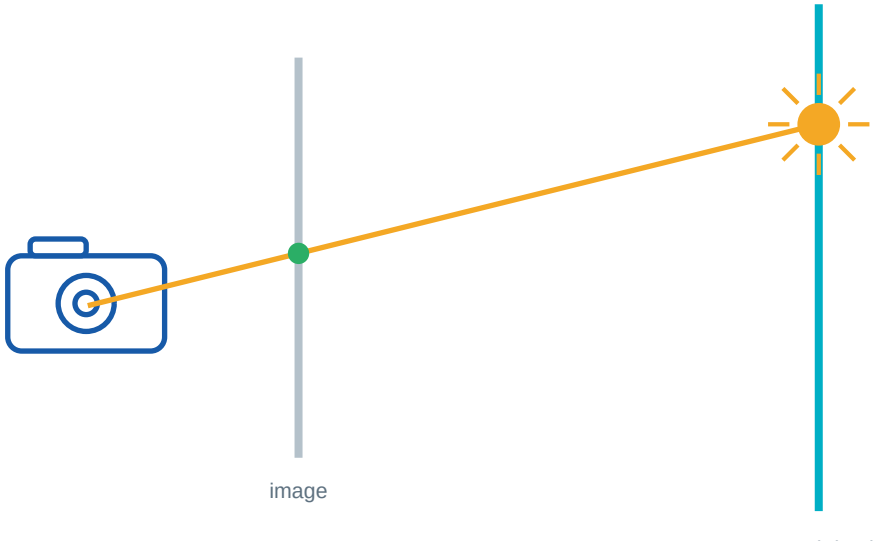
STEP 2A

One camera: direction is measured, distance is assumed

IMPORTANT

One flat picture tells us which direction the light is in, but not how far away it is. GlareTracker asks the user for an assumed depth D (default 120 cm).

Pinhole-camera picture



Pixel offset + camera field of view -> direction

The current formulas

CAMERA FOCAL LENGTH

fx = (image width / 2) / tan(HFOV / 2)

HFOV is the horizontal field of view.

HEADLIGHT POSITION

X = (u - centerX) x D / fx

Y = D

HEIGHT

Z = cameraHeight - (v - centerY) x D / fy

The image Y direction points downward.

Worked example

1280 x 720 image, HFOV 70 deg, point (u=700, v=390), D=120 cm, camera height=19.5 cm -> fx about 914 px -> headlight about (X=7.9, Y=120, Z=15.6) cm.

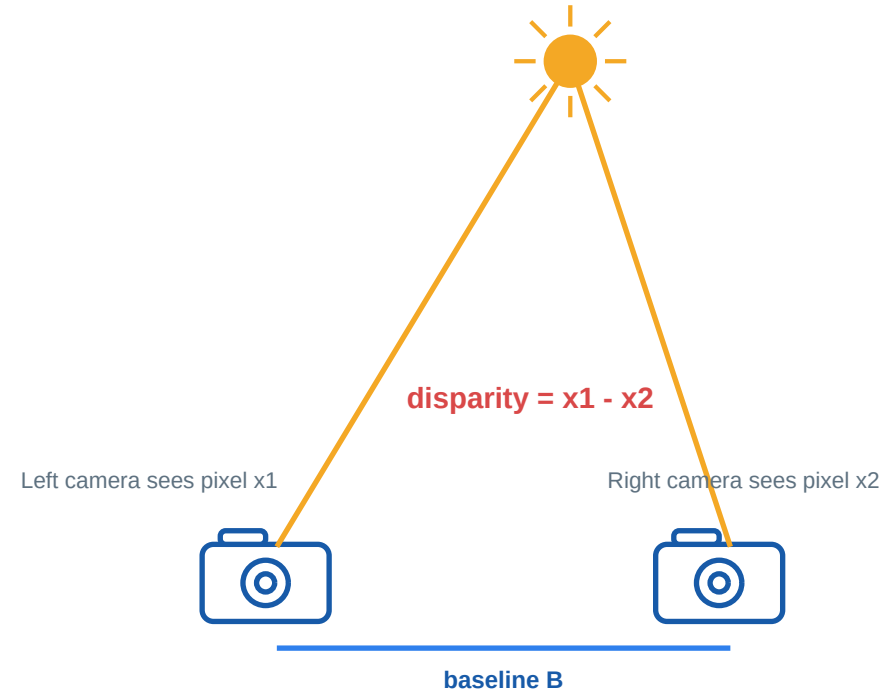
Source: MonocularHeadlightDetectorThread in glare_tracker/headlight_detector.py

STEP 2B

Two cameras: pixel shift reveals distance

Stereo vision works like your two eyes. A nearby object appears to move more between the left and right views. That horizontal movement is called disparity.

Top view



CALIBRATED PATH

Correct lens distortion -> rectify both images -> check similar scanlines -> calculate disparity using the saved camera matrices.

DISTANCE

depth = focalLength x baseline / disparity

Bigger disparity -> closer light

SIDEWAYS

Xcamera = (x1 - centerX) x depth / focalLength

Then shift by half the baseline.

HEIGHT

Zglobal = cameraHeight - Ycamera

Camera pixels point downward; world Z points up.

Fallback example

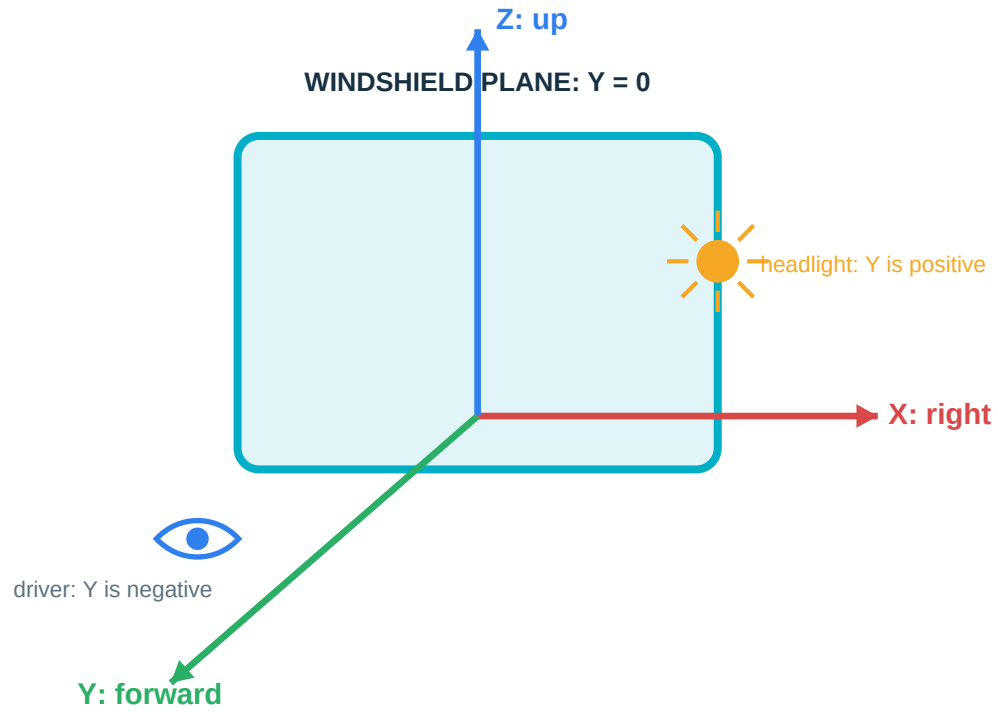
f=900 px, B=25 cm, disparity=75 px -> depth=300 cm. This fallback is an estimate, not a calibrated measurement.

Source: HeadlightDetectorThread and StereoCalibrationResult

STEP 3

Put every measurement into one coordinate system

A position such as (7, 120, 15) is useful only when everyone agrees what X, Y, and Z mean. GlareTracker uses the windshield as the map origin.



Camera-space -> project-space

- Camera X becomes global X (sideways).
- Camera depth becomes global Y (forward).
- Camera-down becomes negative global Z (up).
- The rig is shifted by half its stereo baseline.

HONEST LIMITATION

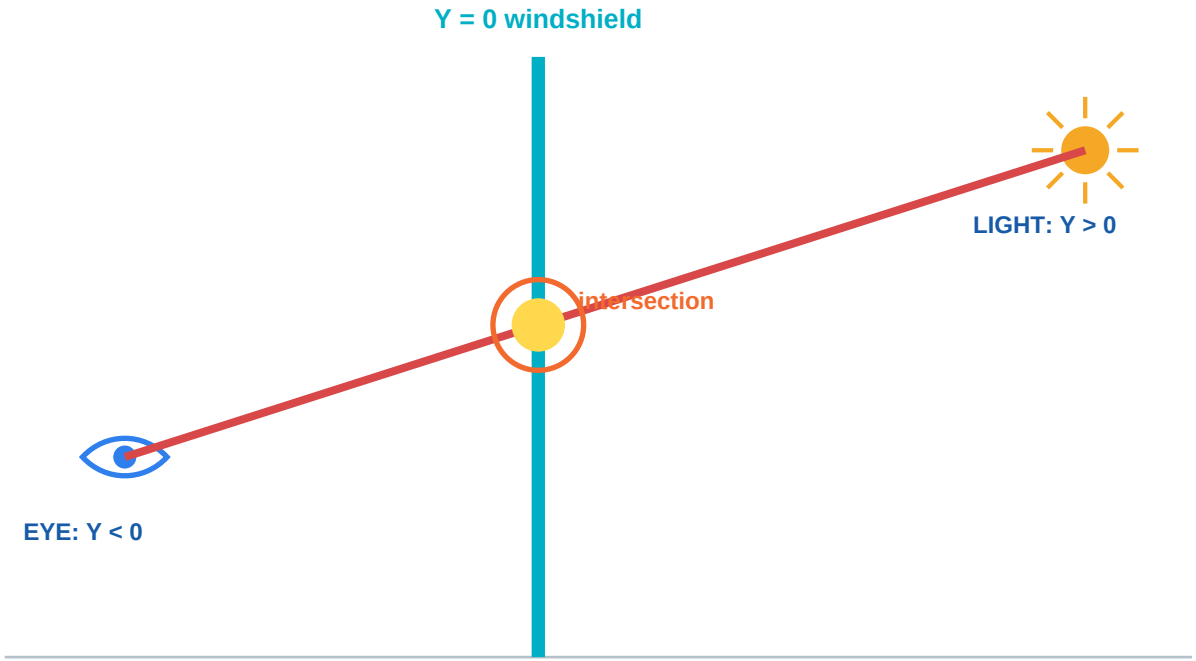
The code assumes the outdoor camera rig is mounted at the windshield top-center and aimed as expected. A measured camera-to-windshield rotation and translation is still needed for measurement-grade physical coordinates.

Coordinate model: X lateral, Y depth, Z height; see PROJECT_STATUS.md and headlight_detector.py

STEP 4

Draw a ray from the eye to the headlight

Now both the eye and headlight have 3D coordinates. The program draws a straight line between them and asks: where does this line cross Y = 0?



HOW FAR ALONG THE LINE?

t = -eyeY / (lightY - eyeY)

Valid when 0 <= t <= 1.

HORIZONTAL HIT

hitX = eyeX + t x (lightX - eyeX)

Same line, same fraction t.

VERTICAL HIT

hitZ = eyeZ + t x (lightZ - eyeZ)

The hit always has Y = 0.

Worked example from the real function

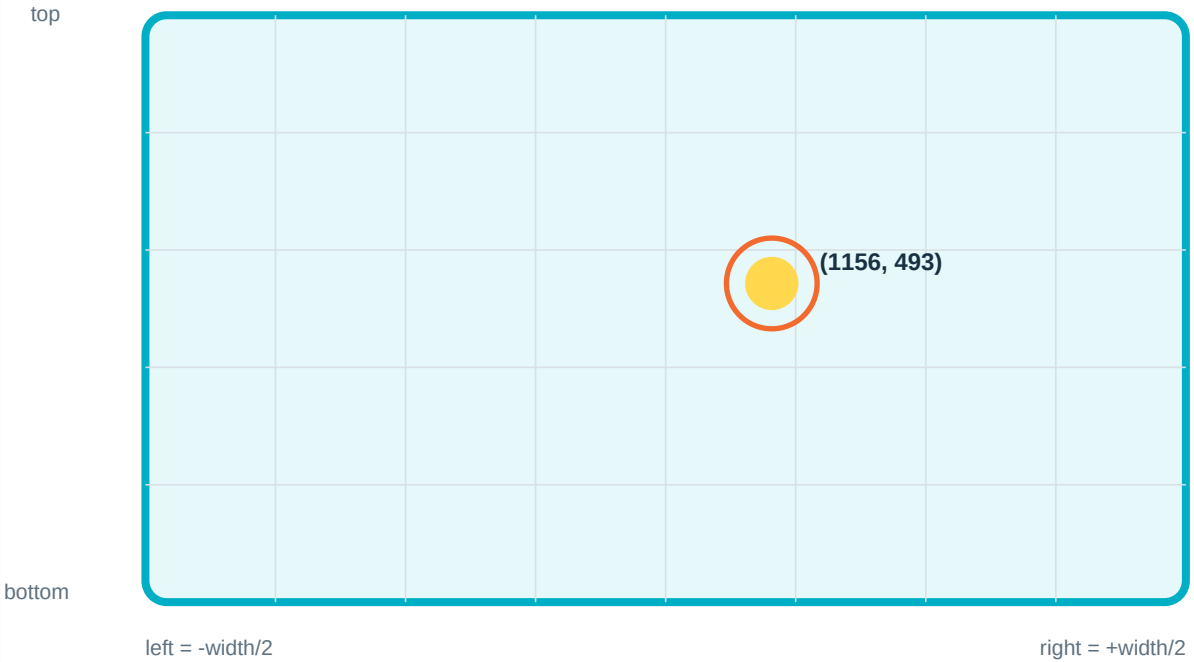
Eye E=(0, -50, 10) cm and light L=(12, 120, 12) cm -> t=50/170=0.294 -> windshield hit I=(3.53, 0, 10.59) cm.

Source: glare_tracker/geometry.py - compute_intersection()

STEP 5

Convert the windshield hit into a screen pixel

The geometry result is measured in centimeters. A real LCD needs a row and column in pixels, so the program scales the point across the panel.



HORIZONTAL PIXEL

px = (hitX + width/2) / width x pixelsX

World center becomes screen center.

VERTICAL PIXEL

py = (height - hitZ) / height x pixelsY

Pixel rows count downward from the top.

Worked example

Panel: 34.5 x 19.5 cm at 1920 x 1080 pixels. Hit: (3.53, 0, 10.59) cm. Result: pixel (1156, 493).

Outside the panel? Return no pixel.

Source: glare_tracker/geometry.py - intersection_to_pixel()

PUTTING IT TOGETHER

The complete idea in one page

1. **See** — Camera captures the road
2. **Detect** — Bright pixels become (u, v)
3. **Locate** — Single or stereo gives X, Y, Z
4. **Connect** — Draw eye-to-light ray
5. **Intersect** — Find where Y becomes zero

What the software already does

- Finds the primary bright light core.
- Supports manual, one-camera, and stereo input.
- Calculates ray-plane intersections for 1-4 lights.
- Checks whether each hit is inside the windshield.
- Can map a hit to an LCD pixel.

What must improve before physical blocking

- Measure each camera pose relative to the windshield.
- Guide calibration through the user interface.
- Use separate left-eye and right-eye rays.
- Filter jitter and predict small system delays.
- Validate error with real cameras and a real LCD.

Four words to remember

Threshold
how bright a pixel must be

Disparity
left-right pixel shift

Ray
a straight directed line

Intersection
where two things meet

Project sources: headlight_detector.py, geometry.py, calibration.py, tracking.py, and PROJECT_STATUS.md